

Année scolaire 2026-2027

Lycée Victor Hugo - Colomiers

Première NSI

AP

**Algorithmes et Programmation
Synthèse**

Thomas Ricart

`ricart.lvh@laposte.net`

`https://thomasricart.github.io`

6 juin 2026

AP01-S

CONSTRUCTIONS ÉLÉMENTAIRES

Définition 1 : Les différents types de variables

- **int** : (entiers naturels) Ex. : `a = 5`
- **float** : (nombres décimaux) Ex. : `b = 3.14`
- **str** : (chaînes de caractères) Ex. : `c = "Bonjour"`
- **bool** : (valeurs booléennes) Ex. : `d = True`
- **list** : (listes) Ex. : `e = [1, 2, 3]`
- **tuple** : (tuples) Ex. : `f = (1, 2, 3)`

Opérations mathématiques de base en Python

+ addition : `5 + 3` donne 8
 - soustraction : `5 - 3` donne 2
 * multiplication : `5 * 3` donne 15
 / division : `6 / 2` donne 3.0 (float)

** puissance : `2 ** 3` donne 8
 % reste de la division entière : `7 % 3` donne 1
 // quotient de la division entière : `7 // 3` donne 2

Propriété 1 : Les chaînes de caractères

Indice	0	1	2	3	4	5	6
Caractère	B	o	n	j	o	u	r
Indice Négatif	-7	-6	-5	-4	-3	-2	-1

</> Code Python

```

mot = 'Bonjour'
mot[0]    # Affiche 'B'
mot[4]    # Affiche 'o'
len(mot)  # Affiche 7

```

Les opérations sur les chaînes de caractères

+ pour la concaténation : `'Bonjour' + ' a tous'` donne `'Bonjour a tous'`
 * pour la répétition : `'Bonjour' * 3` donne `'BonjourBonjourBonjour'`
 [] pour l'indexation : `'Bonjour'[0]` donne `'B'`
 in pour le test d'appartenance : `'on' in 'Bonjour'` donne `True`
 not in pour le test d'appartenance : `'on' not in 'Bonjour'` donne `False`
 len() pour la longueur : `len('Bonjour')` donne 7

Définition 2 : Les booléens

</> Code Python

```

a = True
b = False
c = 6
d = 3

```

</> Code Python

```

a == b  # Affiche False
a != b  # Affiche True
c == d  # Affiche False

```

</> Code Python

```

c <= d  # Affiche False
c > d   # Affiche True
c >= d // 2  # Affiche True

```

Table de vérité du not

Expression	Résultat
not True	False
not False	True

Table de vérité du and

Expression	Résultat
True and True	True
True and False	False
False and True	False
False and False	False

Table de vérité du or

Expression	Résultat
True or True	True
True or False	True
False or True	True
False or False	False

AP02-S

LES FONCTIONS

Gestion des entrées en Python

</> Code Python

```
1 a = input("Entrer une valeur au clavier")      # a sera de type str
2 b = int(input("Entrer une valeur au clavier"))  # b sera de type int
```

</> Code Python

```
1 a = 6
2 print(a)    # Affiche 6
3 print("La valeur de a est", a) # Affiche "La valeur de a est 6"
```

Méthode 1 : Formatage de l'affichage avec f-string et format()

L'idée est de créer un texte à trou que l'on va compléter par des variables.

```
>>> name = 'Paul'
>>> age = 25
>>> print(f'Votre nom est {name} et vous avez {age} ans.')
Votre nom est Paul et vous avez 25 ans.
```

```
>>> print('Votre nom est {} et vous avez {} ans'.format(name, age))
Votre nom est Paul et vous avez 25 ans
```

Définition 1 : Les fonctions

</> Code Python

```
1 def f(x):
2     """ Fonction renvoyant x ** 2 """
3     y = x ** 2      # Calcul de x ** 2
4     return y        # valeur renvoyée: y
```

</> Code Python

```
1 a = 4      # on affecte 4 à la variable a
2 z = f(4)   # on appelle la fonction f avec comme argument 4
3           # et on récupère la valeur renvoyée dans la variable z
```

Remarque : Variables locales et variables globales

- les **variables locales** ne sont connues que dans la fonction
- les **variables globales** sont connues dans tout le programme

</> Code Python

```
1 print(x)    # ERREUR -> x est une variable locale (tout comme y et z)
```

Vérification de la valeur d'une variable

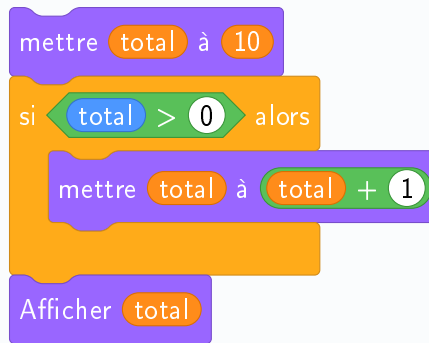
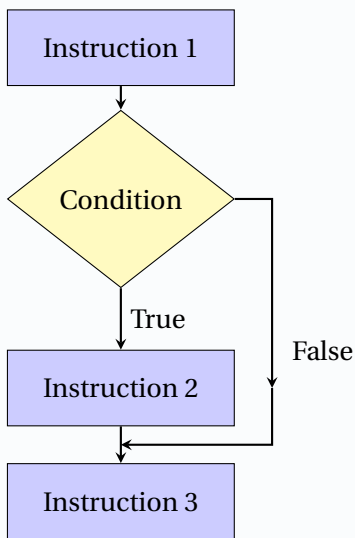
</> Code Python

```
1 assert f(6) == 36      # True -> aucune message
2 assert f(6) == 12      # False -> message d'erreur
3 assert not(f(6) == 12) # True -> aucun message
```

AP03-S

CONDITIONS IF

Les structures conditionnelles IF



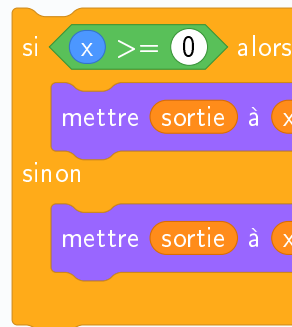
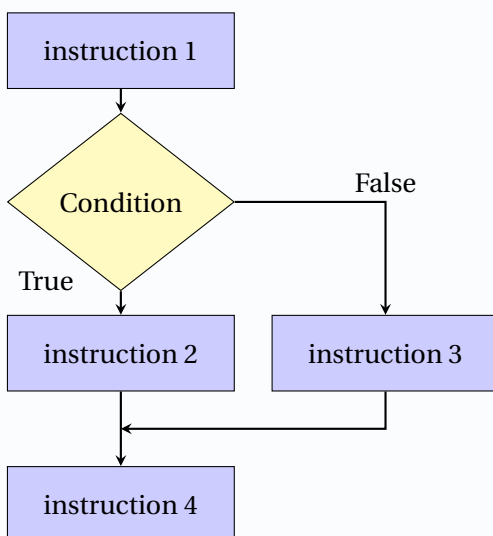
```
# En python
Instruction_1
if condition:
    Instruction_2
Instruction_3
```

```
# En python
total = 10
if total > 0:
    total = total + 1
print(total)
```

- l'Instruction 1 est effectuée
- SI la Condition est True alors l'Instruction 2 est effectuée
- SINON on va directement à l'Instruction 3
- Dans tous les cas l'Instruction 3 est effectuée

La Condition est un test qui est évalué en python par un booléen.

Les structures conditionnelles IF – ELSE



```
# En python
instruction_1
if condition:
    instruction_2
else:
    instruction_3
instruction_4
```

- l'instruction 1 est effectuée
- SI la Condition est True alors l'instruction 2 est effectuée
- SINON l'instruction 3 est effectuée
- Dans tous les cas l'instruction 4 est effectuée

</> Code Python

```
1 def plus_grand2(a, b):
2     ''' Renvoie le plus grand des
3     deux nombres a et b '''
4     if a > b:
5         return a
6     else:
7         return b
8
9 maximum = plus_grand2(5, 10)
10 print(maximum) # Affiche 10
```

</> Code Python

```
1 def plus_grand3(a, b, c):
2     ''' Renvoie le plus grand des
3     trois nombres a, b et c '''
4     if a > b and a > c:
5         return a
6     elif b > a and b > c:
7         return b
8     else:
9         return c
10
11 maximum = plus_grand3(5, 10, 7)
12 print(maximum) # Affiche 10
```

AP04-S**BOUCLES FOR ET WHILE****La boucle FOR en python**

</> Code Python

```
1 for i in range(0, 7):
2     print(i, end=" ")
3 # 0 1 2 3 4 5 6
```

</> Code Python

```
1 for i in range(2, 7):
2     print(i, end=" ")
3 # 2 3 4 5 6
```

</> Code Python

```
1 for i in range(5):
2     print(i, end=" ")
3 # 0 1 2 3 4
```

Itération sur une chaîne de caractères

</> Code Python

```
1 mot = 'Hello'
2 for i in range(0, len(mot)):
3     print(mot[i])
```

</> Code Python

```
1 mot = 'Hello'
2 for lettre in mot:
3     print(lettre)
```

Algorithmes de Recherche dans un itérable

</> Code Python

```
1 def est_present(mot, lettre_cherchee):
2     for lettre in mot:
3         if lettre == lettre_cherchee:
4             return True
5     return False
```

</> Code Python

```
1 def est_present(mot, lettre_cherchee):
2     for i in range(len(mot)):
3         if mot[i] == lettre_cherchee:
4             return True
5     return False
```

Algorithmes de comptage dans un itérable

</> Code Python

```
1 def compte(mot, lettre_cherchee):
2     c = 0
3     for lettre in mot:
4         if lettre == lettre_cherchee:
5             c = c + 1
6     return c
```

</> Code Python

```
1 def compte(mot, lettre_cherchee):
2     c = 0
3     for i in range(len(mot)):
4         if mot[i] == lettre_cherchee:
5             c = c + 1
6     return c
```

Algorithmes de somme

</> Code Python

```
1 def somme(n):
2     ''' Renvoie la somme des entiers de 1 à n inclus'''
3     s = 0
4     for i in range(1, n+1):
5         s = s + i
6     return s
```

Prototypage de la boucle WHILE en python

condition1 est un booléen qui est évalué à chaque tour de boucle. Tant que condition1 True, on effectue les instructions de la boucle. Lorsque condition1 devient False, on sort de la boucle.

</> Code Python

```
1 n = 0
2 u = 1
3 while u < 1000:
4     n = n + 1
5     u = u * 2
6 print(n)
7 print(u)
```

</> Code Python

```
1 mot = 'abracadabra'
2 c = 0
3 trouve = False
4 while c < len(mot) and not(trouve):
5     trouve = mot[c] == 'r'
6     c = c + 1
7 print(trouve)
```

AP05-S

LISTES ET TUPLES

Définition 1 : Les listes

Indice i	0	1	2	3	4	5	6	7
Élément	T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]

Accéder aux éléments d'une liste

</> Code Python

```
ma_liste = [1, 2, 3, 4, 5]
print(ma_liste[0])    # Affiche: 1
print(ma_liste[2])    # Affiche: 3
print(len(ma_liste))  # Affiche: 5
```

Modifier les éléments d'une liste

</> Code Python

```
ma_liste = [1, 2, 3]
ma_liste[0] = 10
print(ma_liste) # Affiche: [10, 2, 3]
print(len(ma_liste)) # Affiche: 3
```

Itérer sur une liste avec une boucle FOR

</> Code Python

```
ma_liste = [4, 8, 7, 9, 4]
for i in range(0, len(ma_liste)):
    print(ma_liste[i])
```

</> Code Python

```
ma_liste = [4, 8, 7, 9, 4]
for elt in ma_liste:
    print(elt)
```

Sommes des éléments d'une liste

</> Code Python

```
def somme(liste):
    somme = 0
    for elt in liste:
        somme = somme + elt
    return somme
#
```

Recherche de maximum dans une liste

</> Code Python

```
def maximum(liste):
    maximum = liste[0]
    for i in range(1, len(liste)):
        if liste[i] > maximum:
            maximum = liste[i]
    return maximum
```

Recherche présence d'un élément dans une liste

</> Code Python

```
def est_present(elt, liste):
    for e in liste:
        if e == elt:
            return True
    return False
```

Vérification liste triée

</> Code Python

```
def est_triee(liste):
    for i in range(0, len(liste)-1):
        if liste[i] > liste[i+1]:
            return False
    return True
```

Liste de listes

</> Code Python

```
ma_liste = [[1, 2, 3], [4, 5], [7, 8, 9, 10]]
print(len(ma_liste))    # Affiche: 3
print(len(ma_liste[1])) # Affiche: 2
print(ma_liste[0])      # Affiche: [1, 2, 3]
print(ma_liste[2])      # Affiche: [7, 8, 9, 10]
print(ma_liste[0][1])    # Affiche: 2
print(ma_liste[1][1])    # Affiche: 5
print(ma_liste[2][0])    # Affiche: 7
```

Sommes dans une liste de listes

</> Code Python

```
def somme(matrice):
    somme = 0
    for liste in matrice:
        for elt in liste:
            somme = somme + elt
    return somme
#
```

Recherche de maximum liste de listes

</> Code Python

```
def maximum(matrice):
    maximum = matrice[0][0]
    for i in range(0, len(matrice)):
        for j in range(0, len(matrice[i])):
            if matrice[i][j] > maximum:
                maximum = matrice[i][j]
    return maximum
```

Quelques méthodes sur les listes

- `append(elt)` : ajoute l'élément `elt` à la fin de la liste
- `pop()` : supprime et renvoie le dernier élément de la liste
- `pop(i)` : supprime et renvoie l'élément d'indice `i` de la liste

</> Code Python

```
ma_liste = [1, 2, 3, 4, 5]
ma_liste.append(6)
a = ma_liste.pop()
b = ma_liste.pop(2)
```

Créer une liste par itération

</> Code Python

```
ma_liste = []
for i in range(0, 5):
    ma_liste.append(i)
print(ma_liste) # [0, 1, 2, 3, 4]
```

</> Code Python

```
ma_liste = []
for i in range(0, 5):
    ma_liste = [i] + ma_liste
print(ma_liste) # [4, 3, 2, 1, 0]
```

Créer une liste par compréhension

</> Code Python

```
ma_liste = [i for i in range(0, 5)]
print(ma_liste) # Affiche: [0, 1, 2, 3, 4]
```

Le hasard et les listes

- La fonction `random.choice(liste)` permet de choisir un élément au hasard dans une liste.
- La fonction `random.shuffle(liste)` permet de mélanger les éléments d'une liste de manière aléatoire.

Le slicing

Le slicing permet de créer une nouvelle liste à partir d'une liste existante en sélectionnant une portion de cette liste.

</> Code Python

```
ma_liste = [1, 2, 3, 4, 5]
print(ma_liste[1:4]) # Affiche: [2, 3, 4]
print(ma_liste[:3]) # Affiche: [1, 2, 3]
print(ma_liste[2:4]) # Affiche: [3, 4]
```

Définition 2 : Les tuples

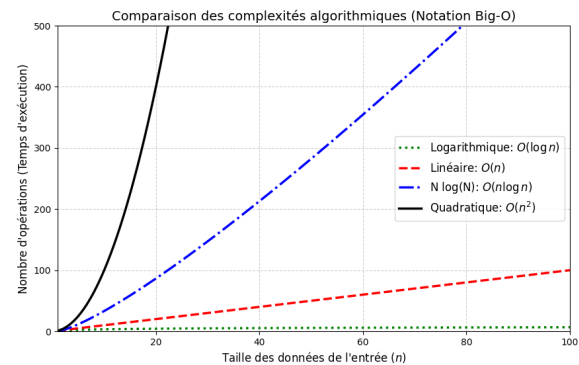
Un **tuple** est une structure de donnée qui contient des éléments ou items identifiés par des indices. Les tuples ont les mêmes caractéristiques que les listes, à la différence que les éléments d'un tuple ne peuvent pas être modifiés après la création du tuple. Un tuple est une structure de donnée immuable.

AP06-S

ALGORITHMES DE RECHERCHE

Définition 1 : Notion de terminaison d'un algorithme

Un algorithme est dit **terminant** s'il s'arrête après un nombre fini d'étapes pour toute entrée donnée.
La condition d'arrêt est appelée **variant de boucle**.



Définition 2 : Le variant de boucle

Un **variant de boucle** est une quantité qui est modifiée à chaque itération d'une boucle et qui permet de garantir la terminaison de l'algorithme. Pour qu'un variant de boucle soit efficace, il doit être :

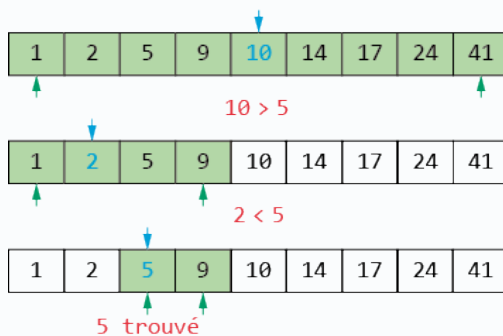
- **positif ou nul** pour que la boucle se poursuive;
- **décroissant** à chaque itération pour que la boucle se rapproche de sa condition d'arrêt.

Définition 3 : Notion de correction d'un algorithme

Un algorithme est dit **correct** s'il renvoie la bonne solution pour toute entrée donnée.

Un **invariant de boucle** est une propriété qui est vérifiée avant et après chaque itération d'une boucle. Si l'invariant de boucle est vérifié, alors l'algorithme est correct.

Définition 4 : Algorithme dichotomique



</> Code Python

```
1 def recherche_dichotomique(val, L):
2     i_deb = 0
3     i_fin = len(L) - 1
4     while i_deb <= i_fin:
5         i_cen = (i_deb + i_fin) // 2
6         if L[i_cen] == val:
7             return True
8         elif L[i_cen] < val:
9             i_deb = i_cen + 1
10        else:
11            i_fin = i_cen - 1
12    return False
```

Définition 5 : Complexité logarithmique

Dans l'exemple précédent, on dira que cet algorithme possède une complexité **logarithmique** notée $O(\log(n))$.

Propriété 1 : Détermination de complexité

- Si un algorithme n'effectue pas de boucle, alors sa complexité est de l'ordre de $O(1)$.
- Si un algorithme effectue une boucle FOR, alors sa complexité est de l'ordre de $O(n)$.
- Si un algorithme effectue k boucles FOR imbriquées, alors sa complexité est de l'ordre de $O(n^k)$.
- Si la taille du problème est divisée par 2, complexité est l'ordre de $O(\log(n))$ (WHILE).